A robust, noise-weighted loss for `HEFTNet`

Plan for the Huber-pull fine-tune: one model first, then the ensemble

For approval before any code is written

October 31, 2026

Abstract

The BNN runs showed that weighting each bin by its POWHEG noise, starting from the L1 solution and using full-data gradients, cuts the mean χ^2 per bin near threshold from 8.4 to 1.9; the bootstrap ensemble showed that an ensemble of L1 models is well calibrated but keeps the threshold bias (`docs/bnn/bnn_run3.pdf`). This plan describes how to obtain both in one procedure, suitable as the headline result of a paper: every network is trained in two stages, the usual L1 training followed by a short fine-tune with the *Huber loss applied to the pulls*, the residuals in units of the POWHEG noise. The first step, described in detail here, tests the fine-tune on a single model for several values of the Huber threshold δ, in a new notebook `code/heft_finetune.ipynb` with a small new module `dihiggs/finetune.py`. If it works, the ensemble notebook gains a fine-tune stage (section 6). Nothing is written until you approve; the decisions are collected in section 8.

The proposal in brief

1. **The loss:** $\ell_\delta(r_k)$, the Huber function of the pull $r_k = (f_k - t_k)/\sigma_k$; quadratic ($\chi^2$) for $|r_k| \le \delta$, linear beyond. σ_k is the POWHEG noise of the bin.
2. **Two stages:** start from a trained L1 model; fine-tune with full-batch gradients, Adam at a small learning rate, saving the parameters with the best validation loss (the same loss, on the validation rows).
3. **No second target column:** each row's noise variance is a separate tensor, aligned with the rows; batches are formed by row *indices*, which pick out the inputs, the targets and the variances together.
4. **The test:** $\delta = 1, 2, 3$ and ∞ (plain χ^2), from `runs/L1Loss`, compared on the 1,000 holdout spectra with the L1 model and the full-data BNN. A few minutes on the 5090.
5. **Success:** threshold χ^2 per bin close to the BNN's 1.9, no loss of accuracy above 555 GeV, fewer negative predictions.

Contents

1 The loss **2**
1.1 Huber on the pulls . . . 2
1.2 What the linear tails buy . . . 3
1.3 The noise of each bin . . . 3

2 Two stages 4

3 Carrying the noise without a second target 4

4 The code 5
4.1 dihiggs/finetune.py 5
4.2 code/heft_finetune.ipynb 5

5 The test 5

6 After the test 6

7 Not in this plan 6

8 Decisions for you 6

1 The loss

1.1 Huber on the pulls

For a training row k with POWHEG value t_k, prediction f_k and noise variance σ_k^2, define the pull $r_k = (f_k - t_k)/\sigma_k$ and minimise the average of

$$\ell_\delta(r) = \begin{cases} \frac{1}{2}r^2, & |r| \le \delta, \\ \delta\left(|r| - \frac{1}{2}\delta\right), & |r| > \delta. \end{cases} \tag{1}$$

The two pieces join smoothly at $|r| = \delta$: same value, same slope. δ is measured in units of the noise: with $\delta = 2$, a bin within two standard deviations of POWHEG contributes its χ^2, and a bin further away contributes only linearly.

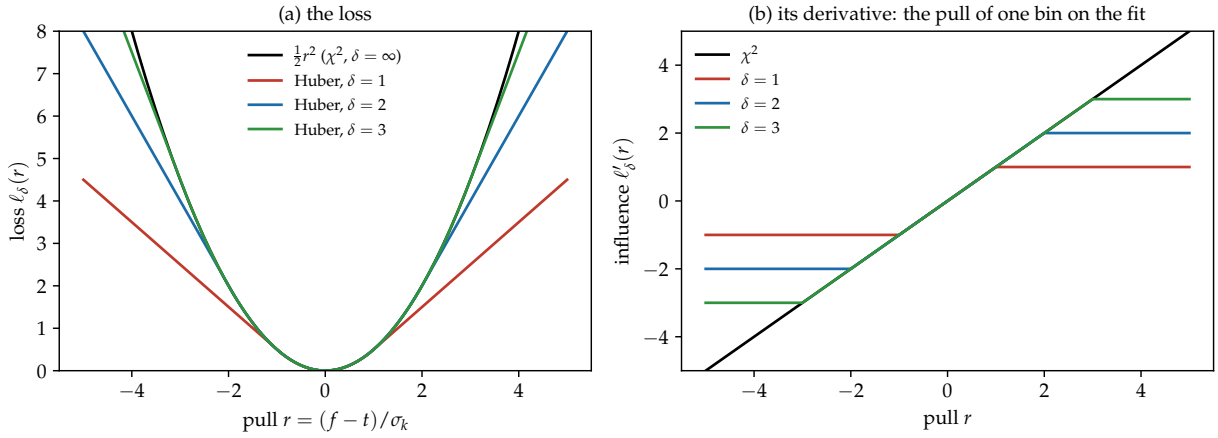


Figure 1: (a) The Huber loss for $\delta = 1, 2, 3$, and $\frac{1}{2}r^2$ (χ^2, $\delta = \infty$). (b) Its derivative, the "influence" of one bin: how hard that bin pulls the fit towards itself. For χ^2 the influence grows without limit with the pull; for Huber it is capped at $\pm\delta$.

Why on the pulls and not on the residuals in pb

Huber applied to $f_k - t_k$ in pb interpolates between MSE and L1, and keeps their weakness: an error of 10^{-4} pb counts the same in every bin, so the large cross sections dominate and the region just above threshold, where the cross section is 10^{-2} of its peak, is neglected. What the BNN showed is that the remedy is to measure each error in units of that bin's noise, as χ^2 does. Huber on the pulls keeps exactly that for the well-modelled bins, which are the vast majority.

1.2 What the linear tails buy

- **Robustness.** A bin with a large pull, from a mismodelled region, one of the ten incomplete POWHEG runs, or an underestimated noise, pulls the fit with a force of at most δ, not $|r_k|$ (fig. 1b). The pulls of the BNN and the ensemble have heavier tails than a Gaussian (`bnn_run3.pdf`, Fig. 3), so this is not hypothetical.
- **Gentler start.** Early in the fine-tune many pulls near threshold are large (the L1 model's threshold χ^2 is 8.4 per bin). With Huber, those bins enter linearly at first and quadratically once they are fitted. The gradient of a bin is capped at δ/σ_k; $1/\sigma_k$ still spans several decades, so this eases the conditioning problem of the plain CHI2 loss without removing it, which is why the fine-tune also starts from the L1 solution (section 2).
- **One story with the BNN.** For $|r_k| \leq \delta$ the loss is the Poisson likelihood of the BNN (a Gaussian with the known variance). The BNN then cross-checks the same fit, not a different one.

Including $\delta = \infty$ (plain χ^2) in the test shows whether the tails matter in practice.

1.3 The noise of each bin

As in `heft_train.ipynb` (before the CHI2 loss was removed) and the BNN notebook: each spectrum behaves as a histogram of about $N_{\text{eff}} \approx 38{,}000$ equal-weight events, estimated from the bin-to-bin jitter of the *training* spectra, and each bin is given the variance of one extra event so that no bin claims to be known better than that:

$$\sigma_k^2 = \left(t_k + \frac{\sigma_{\text{tot}}}{N_{\text{eff}}}\right)\frac{\sigma_{\text{tot}}}{N_{\text{eff}}}, \tag{2}$$

where σ_{tot} is the total cross section of the spectrum to which the row belongs.

Variance from the data or from the model?

Equation (2) uses the observed t_k. A bin that fluctuates down then gets a smaller σ_k, a larger weight, and pulls the fit slightly low, by a relative amount of order $1/n_k$, the inverse of the number of events in the bin (Neyman's χ^2). The alternative is to use the model's own prediction, held fixed during each step (`f.detach()`), in place of t_k (Pearson's χ^2), which removes that bias but makes the weights move during the fit. The bias matters only for bins with very few events. I propose the observed t_k for the test, as in the BNN's Poisson likelihood, so that the two are comparable, with a switch `VARIANCE_FROM = 'data' | 'model'` to try the other (section 8).

2 Two stages

Why not train with this loss from the start. The CHI2 loss failed when trained from random weights with mini-batches of 2,048 (`dihiggs_report.pdf`): median holdout χ^2/N 382 after 20,000 iterations against 3.8 for L1. In the BNN runs the same weighting worked well, starting from the L1 solution and with every gradient computed on all the training rows. The fine-tune reproduces those two conditions.

Stage 1 is the L1 training of `heft_train.ipynb`, unchanged. For the test, stage 1 is the existing model `runs/L1Loss`, the same start as every BNN run, so the comparison with the full-data BNN is direct.

Stage 2, the fine-tune.

- **Full-batch gradients:** each step uses all 274,152 training rows (in chunks of, say, 65,536, whose gradients add up exactly, as in `hmc.Posterior`). About 4 ms per step on the 5090, 0.3 s on a laptop CPU.
- **Adam,** learning rate 10^{-4} halved every 1,250 steps, 5,000 steps in all (switches `FT_LR`, `FT_STEPS`). Adam rescales each parameter's step by its own gradient history, which helps with the spread of scales.
- **Validation:** every 50 steps the same loss is computed on the validation rows of `heft_train.ipynb`, and the parameters with the lowest value are kept; the loss history is written to `dihiggs_losses.csv` as usual.
- **Divergence guard:** if the training loss becomes non-finite, the fine-tune stops and keeps the best parameters so far.

3 Carrying the noise without a second target

The loss needs σ_k for every row of a batch, and the noise variance must not be a second target column. The fine-tune therefore does not use `drwho`'s `DataLoader`, which hands out (inputs, targets) only. Instead:

1. three tensors, on the GPU, aligned row by row: inputs `X` [N, 6], targets `T` [N], noise variances `V` [N];
2. a batch is a set of row *indices* `idx`; the batch is then `X[idx]`, `T[idx]`, `V[idx]`;
3. with full-batch gradients, the "batches" are simply consecutive chunks of the rows, and `idx` is a slice.

The target remains the cross section alone; the variance is a known property of each row, handed to the loss beside it, exactly as `hmc.PoissonLikelihood` receives it. `V` is computed once, from Eq. (2), from the training file itself; for `VARIANCE_FROM = 'model'` only the $\sigma_{\text{tot}}/N_{\text{eff}}$ part is stored and the t_k in Eq. (2) is replaced by the detached prediction.

4 The code

4.1 `dihiggs/finetune.py`

A small module, commented in the style of `hmc.py`, importing nothing from the rest of `dihiggs` except what is passed in:

function / class	what it does
`estimate_n_eff(df)`	N_{eff} from the bin-to-bin jitter of the spectra in a training data frame (the code now repeated in three notebooks)
`noise_variance(df, n_eff)`	Eq. (2) for every row; also returns $\sigma_{\text{tot}}/N_{\text{eff}}$
`HuberPull(delta)`	the loss (1) of predictions f, targets t, variances v, averaged; `delta=None` gives $\frac{1}{2}r^2$
`finetune(model, X, T, V, Xv, Tv, Vv, ...)`	stage 2: full-batch Adam with the learning-rate steps, validation every 50 steps, best-parameter saving, loss file; returns the loss history

4.2 `code/heft_finetune.ipynb`

In the style of `heft_train.ipynb` (header, Colab cell, imports, device, data with its SHA-256), then:

1. **Introduction:** the loss, the pulls, the two stages (condensed from sections 1 and 2).
2. **Switches:** `START = 'runs/L1Loss/dihiggs_params.pth'; DELTAS = [1, 2, 3, None]` (`None` = χ^2); `VARIANCE_FROM = 'data'`; `FT_STEPS = 5000`; `FT_LR = 1e-4`; `SEED = 42`.
3. **The split** of `heft_train.ipynb`; N_{eff} and the variances.
4. **The fine-tunes:** one per value of δ, each from the same starting model, saved in `runs/FT_huber_d1/`, `FT_huber_d2/`, `FT_huber_d3/`, `FT_chi2/`: an ordinary run folder (configuration with δ, N_{eff}, the schedule and the starting file; `dihiggs_params.pth`; the loss file), so that `heft_test.ipynb` and `heft_export.ipynb` work with any of them.
5. **Results on the holdout spectra** (section 5).

5 The test

On the 1,000 holdout spectra, for the starting L1 model, each fine-tuned model and, if its folder is present, the full-data BNN mean:

quantity	why
mean χ^2 per bin, 255–555 GeV	the target of the exercise (L1: 8.4; BNN: 1.9)
mean χ^2 per bin, above 555 GeV	must not get worse (L1: 1.20; BNN: 1.12)
median χ^2/N; fractions above 2 and 5	overall accuracy, and the bad spectra
mean |residual| in pb	what L1 optimises; a small loss here is expected
negative predictions	physical sanity
χ^2 per bin against m_{hh} (figure)	where the gains and losses are
the loss curves (figure)	that the fine-tune converged, without overfitting

What counts as success. A value of δ for which the threshold χ^2 per bin is close to the BNN's 1.9, the χ^2 above 555 GeV is no worse than the L1 model's, and negative predictions are fewer. If $\delta = \infty$ does as well as finite δ, the robustness argument still favours a finite value; if it does clearly worse, the tails matter. If no value succeeds, the next things to try are a smaller learning rate, more steps, and `VARIANCE_FROM = 'model'`.

Cost. Four fine-tunes of 5,000 full-batch steps: about 1.5 minutes on the 5090 (about two hours on a laptop CPU, so the 5090 is the place for it).

6 After the test

1. **The ensemble with the chosen δ.** `heft_train_ensemble.ipynb` gains a switch `FINETUNE_DELTA` (default `None`: no fine-tune, as now). With a value, each member, after its L1 training, is fine-tuned on *its own* bootstrap rows, and saved as before; the stage-1 parameters are kept as `dihiggs_params_L1.pth`. The fine-tune adds well under a minute per member on the 5090. Then the bootstrap and `'seeds'` ensembles are trained this way.
2. **The test of the ensemble** is that of `bnn_run3.pdf`: accuracy and the pull. Success: as accurate as the BNN, as well calibrated as the L1 ensemble.
3. **The BNN cross-check:** the full-data Poisson BNN with the fixed warm-up, started from the fine-tuned model rather than the L1 model (shorter descent).

7 Not in this plan

- No change to `heft_train.ipynb`, `heft_test.ipynb`, `hmc.py` or the data. The three notebooks that estimate N_{eff} keep their own code for now; they can later call `finetune.estimate_n_eff`.
- The changes to `heft_train_ensemble.ipynb` of section 6, which wait for the result of the test.
- Exact per-spectrum event weights w_j (recoverable from the quantization of the spectra) instead of the N_{eff} estimate: a later refinement.
- Nothing is committed to git.

8 Decisions for you

Decisions before I proceed

1. **Huber on the pulls** (not on the residuals in pb), with $\delta \in \{1, 2, 3, \infty\}$ in the test. Agreed?
2. **Two stages,** starting the test from `runs/L1Loss`. Agreed?
3. **Full-batch Adam,** 10^{-4} halved every 1,250 steps, 5,000 steps, best validation loss kept. Agreed, or would you prefer large mini-batches?
4. **The noise as a separate tensor** indexed with the rows (not a target column). Agreed?
5. **Variance from the data** for the test, with the `'model'` option available. Agreed?

6. **A new module and notebook,** `dihiggs/finetune.py` and `code/heft_finetune.ipynb`; folders `runs/FT_huber_d<delta>/`, `runs/FT_chi2/`. Acceptable?
7. **Location of this document:** `docs/huber/huber_plan.pdf`?